CS175 Project Progress Report

Atharv Attri	Adhith Jacob	Shreya Nakum
aattri@uci.edu	adhithj@uci.edu	snakum@uci.edu
71872115	66072553	17143467

16 August 2026

1 Project Summary

The question from the proposal hasn't changed: Polymarket's fee structure on fifteen-minute BTC Up/Down markets is large enough that a well-calibrated probability estimate is not the same thing as a profitable trading policy, and we are still trying to find out whether a state-dependent policy can do something a fixed threshold cannot. The state space, action space, reward, and three-baseline comparison described in the proposal are unchanged.

Since the proposal we have built the dataset pipeline, the cost model, both non-RL baselines, and the Q-learning agent, and we have evaluation results for all of them on their held-out splits. The RL agent and the two baselines were built somewhat independently and do not yet share a backtest harness, so the single head-to-head comparison the proposal describes is not finished. Every policy tested so far, in both tracks, loses money after fees. What is left is reconciling the two harnesses so the comparison is apples to apples, running the hyperparameter sweep we have not started, adding the slippage modeling the RL environment is still missing, and reading the test split exactly once, at the end, as planned. Whether we get to the paper-trading moonshot depends on how much of that is done in the remaining two weeks.

2 Approach/Algorithm

2.1 Data Acquisition and Candle Construction

All data originates from two fetchers against Polymarket's public APIs: `data/fetch_btc_updown_15m.py` pulls 1-minute sampled quotes from the CLOB's `prices-history` endpoint, and `data/fetch_btc_updown_15m_trades.py` pulls every individual fill from the data-api `trades` endpoint and aggregates it into 15-second OHLCV candles. Both pulls are cached to disk and resumable, and cover 90 days / 8,608 consecutive 15-minute BTC Up/Down markets (2025-11-28 to 2026-02-26), totaling 44.9 million trades and $795.3M of notional volume.

The two sources are not interchangeable, and the gap is the reason the rebuilt candles rather than the minute file drive every fill in this project. Measured against the trade tape, the minute-sampled feed's price at minute m best matches the market at minute $m-1$: correlation against true trade price peaks at 0.9942 at exactly 60 seconds of lag and falls off cleanly on either side. On a market that only lives 15 minutes, a 60-second lag is 1/15 of the contract's entire life, and any policy that reads current price off the minute file is actually reading the market's state from a minute ago - which may falsely look like predictive signal in a backtest.

Two data-quality issues are handled at construction time rather than left for downstream code to trip over. Prices outside $[0, 1]$ are impossible for a binary market but do appear in the raw feed (one fill in this pull was priced at 4.5137); these are dropped during aggregation, and every one of the 860,800 resulting candles is verified to satisfy low $\leq$ {open,close,vwap} $\leq$ high and lie in $[0.001, 0.999]$. Separately, the trades API hard-caps pagination at 15,000 fills per market with no time filter, which truncates the pre-open tape (not the live window) for the 18 highest-volume markets in the set; these are flagged with a `truncated` column rather than dropped, since excluding them would bias the sample away from exactly the markets with the most trading activity.

2.2 Cost Model, Features, and Data Splitting

Three pieces of shared infrastructure sit underneath the models, and we describe them first because the comparisons in Section 4 are only meaningful if every policy is scored against the same costs and the same data.

The cost model (`sim/execution.py`) implements Polymarket's taker fee, $\text{fee} = C \cdot r \cdot p(1-p)$ for C shares at price p, with $r = 0.07$ for the crypto category. Two properties of this fee drive the whole project. It is symmetric in p and peaks at the money, so trading a coin-flip contract is roughly six times more expensive than trading one priced at 0.90; and

redemption at settlement is free, so holding to resolution is strictly cheaper than closing early. The module also applies adverse slippage, displacing each fill from the candle mid by a configurable fraction (default 0.25) of that candle's high–low range, always in the direction that hurts the trader. This matters because the 15s candles show a median high–low range of 0.04 on traded candles, which is bid-ask bounce in a wide book rather than noise: a round trip pays that spread twice on top of the two fees. Feature construction (`BaselineModels/features.py`) produces one row per (market, candle), with every feature computed only from candles at or before that index. Features include the contract price and its logit, returns over lookbacks of 1, 4, 8, and 16 candles, realized volatility over 8- and 16-candle windows, per-candle trade flow over 4- and 16-candle windows, and time remaining. Each row also carries the *next* candle's open, high, and low, so an action triggered by a row fills at a price that was not among that row's own inputs. Two leakage traps are handled explicitly: the `volume` column is the market's *final* total stamped onto every row and is excluded from the feature allowlist, and the minute-sampled dataset lags the true tape by roughly 60 seconds, making it safe as an input but fatal as an execution price, so fills always come from the 15s candles.

Data splitting (`BaselineModels/data_loader.py`) is strictly temporal: splits are cut at 70/15/15 of the calendar span on each market's `start_ts`, never on an individual row's timestamp, so no market straddles a boundary. Every load asserts the returned markets are disjoint from the other splits, and the test split refuses to load without an explicit `allow_test=True` argument. Our proposal named lookahead bias as the single most likely way this project produces a fake result, and the intent was that the loader enforce the cutoff rather than relying on us to be careful by hand.

2.3 Baseline 1: XGBoost with a Threshold Policy

The XGBoost baseline is split into two stages that are kept deliberately separate.

The first stage is a pure forecast: a gradient-boosted tree ensemble predicting $P(\text{Up})$ from the features at candle c, trained with up to 2,000 boosting rounds and early stopping after 50 rounds without validation improvement. The thing this model must beat is not a coin flip but the market price itself, which is already a calibrated, incentive-backed probability available for free. Because the market price is one of the input features, the model can score respectably by simply echoing it, so we evaluate against the market as a competing forecaster rather than against a naive prior.

The second stage is a fixed, intentionally unintelligent policy: take a position whenever $|\hat{p}_t - p_t^{\text{mkt}}| > \theta$, then hold to resolution, since selling early pays the taker fee a second time while settlement is free. The threshold θ is chosen by sweeping nine candidates from 0.01 to 0.20 and selecting the one maximizing validation PnL after fees — not forecast accuracy, since a well-calibrated model with no tradeable edge is worthless here. A candidate is eligible only if it produces at least 30 trades, which prevents the sweep from selecting a threshold so tight that it wins on a handful of lucky trades. Keeping the policy fixed and dumb is what makes it a baseline for the RL agent: both consume the same signal, differing only in that the baseline's entry rule is a constant while the agent's is state-dependent.

All policies run through a single backtest harness (`BaselineModels/backtest.py`) that owns the cost model and enforces the execution rule: a decision made on candle c fills against candle $c + 1$. Policies are plain functions handed a `Step` object carrying the current position and no reference to the underlying DataFrame, so a policy is structurally unable to see a price it has not yet been allowed to see.

2.4 Baseline 2: Buy-and-Hold

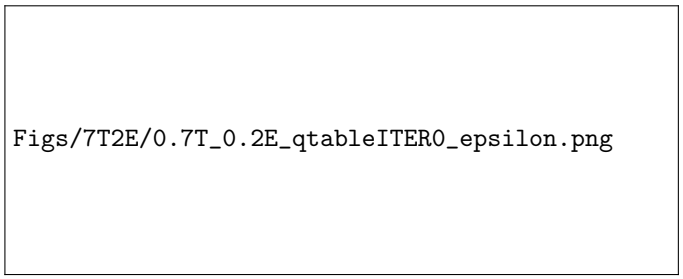
Buy-and-hold takes a position at the first live candle on whichever side the market favors and holds to settlement, never exiting early. It therefore pays exactly one taker fee and one increment of entry slippage, and collects the free redemption at resolution. It is the sanity check: any system that cannot beat simply accepting the market's opening price is not adding anything.

One implementation detail turned out to be load-bearing rather than cosmetic. Because the order book quotes in whole cents, 8.2% of validation markets open at exactly 0.500, so the tie-breaking rule selects the side for roughly one market in twelve — and does so at precisely the price where the taker fee is at its maximum. We follow the proposal's specification of taking the favored side, resolving ties to Down, but the harness exposes the alternatives of resolving to Up or of declining to trade, and we report which was used because the choice is not neutral. A further 13 validation markets have no candle at index 0; these enter at their first available row rather than being dropped, so that the market sample is identical across all policies and the comparison stays like-for-like.

2.5 Baseline 3: Rule-Based Momentum Flip

Separately from the XGBoost/backtest-harness track, a second pair of rule-based policies (`strategies/`) was run over the full 15s tape - all 8,555 markets with a complete live window, not the temporal validation split - as an independent sanity check on whether any simple signal survives realistic execution costs. This track uses its own flat per-share slippage cost, distinct from the Polymarket taker-fee formula in `sim/execution.py`, so its results are not directly comparable to Table 4 without reconciling the two cost conventions (see Section 3.3).

Figure 1: 70/20 Split Q-Learning Agent Characteristics in First Iteration



(a) ϵ-decay

(b) Rewards Across Training

(c) Training Action Frequencies

(d) Evaluation Action Sequences Heatmap

momentum_flip waits until either side of the market trades at or above a 0.55 threshold and buys it; whenever the *other* side subsequently crosses 0.55, it sells the full position and rolls the proceeds into that side; whatever is held at the bell goes to resolution. As with every other policy in this project, a signal read off candle i's close fills at candle $i+1$'s open, so nothing trades on a price it could not yet have seen. **buy_and_hold_down** is a second, independent implementation of the buy-and-hold baseline - buy Down at the open of candle 0 and hold to resolution - run here on the full dataset rather than the validation split, as a cross-check against the buy-and-hold result reported in Section 4.1.

2.6 Reinforcement Learning Algorithm

As stated in the proposal, we treated each market's price path, from start to finish, as an episode. This episode was detailed in an environment.

We framed the trading problem as an Markov Decision Process (MDP) and solved it with tabular Q-learning. Each episode is a single 15-minute Polymarket up/down market, discretized into 60 fifteen-second candles, so an episode terminates after at most 60 steps regardless of the agent's actions. At each step t the agent observed a state $s_t = (\text{price}_{\text{bucket}}, \text{time}_{\text{bucket}}, \text{position}, \text{pnl}_{\text{bucket}})$, the current market price discretized into 10 buckets, the candle index into 60 time buckets, the agent's position into 3 categories (`FLAT`, `LONG_UP`, `LONG_DOWN`), and, when the agent holds a position, its unrealized profit and loss (PnL) relative to entry is discretized into 5 buckets (2 loss, 1 neutral, and 2 profit). So, we have state space of size $10 \times 60 \times 3 \times 5 = 9000$ discrete states. The action space contains 4 actions: `HOLD`, `BUY_UP`, `BUY_DOWN`, `SELL`. To make sure that these actions are made properly, the actions are masked by position: when flat, the agent may hold or open a new position in either direction; when holding a position, it may only hold or close it. This masking is enforced both at action-selection time and inside the Bellman backup, so the agent never selects and never bootstraps off of an invalid action.

The agent is a standard Q-learning agent, storing $Q(s,a)$ as a $10 \times 60 \times 3 \times 5 \times 4$ array initialized to zero. Actions are selected within an ϵ-greedy policy over the valid action set, with ϵ starting at 1.0 and decaying geometrically by a factor of 0.995 after every training episode down to a floor of 0.02, so the agent explores heavily early on and is almost fully exploitative for the bulk of training. Ties among equal Q-values are broken uniformly at random instead of a default action, which prevents the agent from getting stuck favoring `HOLD` while all Q-values are still zero. After each transition, the agent performs the standard off-policy Temporal-Difference (TD) update:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\Big[r_t + \gamma \max_{a' \in \mathcal{A}_{valid}(s_{t+1})} Q(s_{t+1}, a') - Q(s_t, a_t)\Big] \tag{1}$$

with learning rate $\alpha = 0.1$ and discount factor $\gamma = 0.99$, where the max over next-state Q-values is restricted to the actions that are valid from s_{t+1}. The $\gamma \max_{a'} Q(s_{t+1}, a')$ term is dropped on terminal transitions because, in the trading environment, at the last candle of each market, the position gets forced-settled to a payout of either \$1 or \$0. That reward reflects the true final outcome. There's no future to be estimated so the $\gamma \max_{a'} Q(s_{t+1}, a')$ term would just be adding noise.

The reward function is the agent's realized dollar PnL, net of Polymarket's taker fee, $\text{fee}(p) = 0.07 \cdot p \cdot (1-p)$, charged on the traded price p. Opening a position (`BUY_UP` and `BUY_DOWN`) immediately causes a fee-based penalty and selling (`SELL`) gives a reward equal to the gross price change since entry minus the exit-side fee. If the agent is still holding a position when the episode ends, the position is force-settled at the market's binary resolution (\$1 or \$0), with no additional fees. Each market is played out independently as one episode, and the agent is trained for 6,789 episodes per run, repeated over 30 independent runs for variance. Models were trained on 70% and 80% of the data collected and tested on the remaining 20%, which was always fully held out. Evaluation metrics include win rate, average and total PnL, Sharpe ratio, maximum draw-down, turnover, and the fraction of gross PnL consumed by fees.

There's one change from our original proposal. We initially charged the full round-trip fee in the beginning when a position was opened so that holding a settlement, which had no separate exit fee, was still accounted for. However, this had the effect of charging the fee again on top of that when a position was later closed on purpose, effectively taxing an early exit 1.5 times more than holding to expiry. This made the agent use `SELL` a lot less, especially in early runs. So, we corrected the reward function to have one taker fee in the beginning and another at the end, if needed.

3 Schedule/Milestones, Remaining Goals and Challenges

3.1 Schedule/Milestones

In our proposal we laid out the following schedule:

1. Week 1-2: Build the dataset pipeline for getting and cleaning historical Polymarket data.
2. Week 3: Build the simulation environment, including the fee and slippage modeling, that the RL will be trained and evaluated against.
3. Week 4: Implement and evaluate the XGBoost and buy and hold baseline models.
4. Week 5: Train and tune the Q-learning agent and run comparisons against the baseline.
5. Week 6: Finalize evaluation results and write the paper and presentation.

Based on this schedule, we are on track. We have finished the dataset pipeline: two fetchers pull 1-minute sampled quotes and true 15-second trade candles from Polymarket's CLOB and data-api endpoints respectively (Section 2.1), covering 90 days and 8,608 markets, with pagination truncation and out-of-range ticks handled at construction time rather than left for downstream code.

The week 3 simulation environment is completed (`make_environment.py`), as it implements a full state/action.reward MDP described in Section 2, including a fee model that matches Polymarket's actual taker-fee schedule ($\text{fee}(p) = C \cdot \text{feeRate} \cdot p \cdot (1-p)$, $\text{feeRate} = 0.07$) for crypto markets. However, we have not yet implemented slippage modeling, which was part of the original Week 3 milestone; the environment currently assumes every trade fills exactly at the observed `close` price with no bid-ask spread or market impact, which likely makes the simulated results somewhat more optimistic than real trading would be.

The Week 4 baseline milestone is met. The supporting infrastructure is complete and covered by a suite of 146 unit tests spanning the cost model, feature construction, temporal splitting, the backtest harness, and the metrics: the XGBoost model is implemented and trained, the buy-and-hold baseline is implemented, and both have been run end to end on the validation split alongside the no-trade floor, producing the results in Section 4. Two deliberate sample cuts apply: 18 training and 1 validation market are dropped for insufficient warmup, giving effective counts of 5,914 and 1,343 markets. The test split remains entirely unread.

The results are negative, and informative. Our forecast does not beat the market price, and neither trading policy is profitable after fees. We regard this as the milestone having been achieved rather than missed, since the purpose of a baseline is to establish what the RL agent must clear, and we now know that bar precisely.

The week 5 (Q-learning training) is functionally complete in that we have a working agent trained and evaluated across 30 independent runs for two different train/validation splits, but "tuning" so far has been limited to the default hyperparameters ($\alpha = 0.1$, $\gamma = 0.99$, epsilon schedule as described); we have not yet run a systematic sweep over learning rate, discount factor, or state discretization granularity, which we still consider part of that week's goals.

3.2 Remaining Goals

One of the main goals for the Q-learning model left is to add slippage modeling but it's complicated by the fact that our current dataset only has the `close` prices per candle. For the Q-learning agent, four items remain

1. **Slippage modeling.** The environment currently fills every trade exactly at the observed `close`, with no spread or market impact, so our reported PnL is optimistic relative to live trading. This is complicated by the fact that the episodes we feed the environment carry only the closing price per candle. The 15s candle dataset does retain `open`, `high`, and `low`, so the natural fix is to widen the episode frame to include them and fill at a price displaced from the mid by a fraction of the candle's high–low range. Given that the median high–low spread on a traded candle is 0.04, this is not a small correction, and we expect it to push results further negative.

2. **Hyperparameter sweep.** Tuning so far has been limited to defaults ($\alpha = 0.1$, $\gamma = 0.99$, and the ϵ schedule described in Section 2). We plan a sweep over learning rate, discount factor, and the granularity of the price and PnL discretiztionss.

3. **Reward shaping for early exit.** The current policy almost never uses `SELL`, which is rational under our fee structure but means three of the four actions are effectively unused. We want to establish whether this is the correct equilibrium or an artifact of the agent never accumulating enough experience in the position-holding states to value exiting.

4. **Head-to-head comparison under identical costs.** The agent's numbers are only interpretable next to the no-trade floor, buy-and-hold, and the XGBoost threshold policy, scored on the same markets with the same fee model. Reporting the test split, which remains untouched, comes last.

1. Produce the four-way comparison — no-trade floor, buy-and-hold, XGBoost with its selected θ, and the Q-learning agent — on identical markets under identical fees and slippage. This is currently blocked by the two tracks using different evaluation protocols.

2. Reconcile the two tracks: move the RL agent onto the temporal loader and the shared cost model, and settle on a single definition of the fee-to-gross-PnL ratio across both (see Section 3.3).

3. Investigate whether any forecast can improve on the market price, or establish that none can. If the honest answer is that the price is not beatable at this horizon, the project's contribution becomes the cost analysis rather than the forecast.

4. Score the test split exactly once, at the end.

3.3 Challenges

One of the main challenges for the Q-learning model is that our Q-table has 9,000 states times 4 actions and with only 6,789 training episodes per run, many state-action pairs are visited only a few times. If a hyperparameter sweep doesn't improve convergence, we may need to coarsen the state discretization or switch to a DQN. **The forecast does not beat the price, and this is now measured rather than anticipated.** Our model's validation log loss is 0.4554 against the market's 0.4548, with Brier score, expected calibration error, and AUC all likewise marginally worse. A bootstrap paired over 1,343 markets puts the difference at -0.000721 with a 95% confidence interval of $[-0.001926, +0.000463]$, and our model was ahead in only 11.7% of resamples. AUC broken out by time remaining matches the market to three or four decimal places at every horizon. The honest reading is that the model has learned to reproduce the market price and nothing beyond it, which is exactly the failure the comparison against the market was built to detect. Whether a genuinely independent signal exists at this horizon is the open question for the remainder of the project.

Threshold selection hit a boundary rather than an optimum. Across the eligible grid, validation PnL is monotone decreasing in θ, so the selector returned the smallest candidate on the grid. This is a boundary selection, not a tuned optimum, and we report it as such. The two thresholds that showed positive PnL were correctly excluded by the minimum-trade constraint, having produced 4 and 1 trades respectively — a useful demonstration that the constraint is doing real work.

The two tracks compute fee drag differently, and the gap is enormous. Our metrics take fees as a fraction of net gross PnL; the RL track's environment clamps the denominator to positive PnL only, so losing trades contribute nothing to the denominator while their fees remain in the numerator. On identical trades and identical fees these two conventions differ by a factor of 21.5: 147.3% against net gross versus 6.9% against positive-only. Before the final report places both models in one table, we must agree on one definition, or the comparison will be meaningless.

4 Evaluation Plan

4.1 Baseline Models

We evaluate the baselines at two levels, because a forecast can be excellent and a policy built on it still unprofitable, and the two failure modes call for different responses. All figures below are computed on the *validation* split of 1,343 markets, after fees and slippage. The test split has not been read.

Forecast Quality

At the forecast level the question is not whether our probability estimate is accurate in absolute terms but whether it is more accurate than the price we would be trading against. Table 1 reports both, with a constant 0.5 forecaster included for scale.

	Our model	Market price	Constant 0.5
Log loss	0.4554	**0.4548**	0.6931
Brier score	0.1528	**0.1527**	0.2500
Expected calibration error	0.0126	**0.0095**	0.0154
AUC	0.8590	**0.8591**	0.5000

Table 1: Forecast quality on 80,283 validation rows across 1,343 markets, base rate 0.485. Lower is better for log loss, Brier, and ECE; higher is better for AUC. The best value in each row is bolded — it is the market's in every case.

The model does not beat the market on any of the four metrics; it is marginally worse on all of them. A bootstrap resampled over the 1,343 markets — rather than over rows, since the 60 candles within a market share a single label and a row-level interval would be roughly $\sqrt{60}$ too narrow — places the log-loss improvement at -0.000721 with a 95% confidence interval of $[-0.001926, +0.000463]$. The interval includes zero and our model was ahead in only 11.7% of resamples, so the fair statement is that our forecast is indistinguishable from the market price rather than reliably worse than it. Table 2 makes the point more sharply: AUC broken out by time remaining matches the market to three or four decimal places at every horizon.

Candles remaining	n	Our model	Market price
0–4	5,372	0.9911	0.9911
4–8	5,372	0.9749	0.9749
8–12	5,372	0.9561	0.9566
12–16	5,372	0.9371	0.9372
16–20	5,372	0.9201	0.9201
20–30	13,419	0.8768	0.8769
30–40	13,388	0.8131	0.8135
40–60	26,616	0.6948	0.6951

Table 2: AUC by time remaining. Both forecasters degrade identically as the horizon lengthens, and the model never separates from the price at any horizon.

Two further details support the reading that the model has learned to copy the price. Early stopping fired after 153 of a possible 2,000 boosting rounds, with the best iteration at 102 — the model stops improving almost immediately, which is what one expects once it has found p^{mkt} and nothing beyond it. And its calibration error, while small in aggregate, is not uniform: it is under-confident below 0.5 and over-confident between 0.5 and 0.8, with individual bins off by roughly two percentage points in opposite directions, so an ECE of 0.0126 understates the per-bin miscalibration (Figure 2).

Threshold Selection

Across the eligible range, PnL is monotone decreasing in θ, so the selector returned the smallest candidate on the grid. This is a boundary selection rather than a tuned optimum: there is no interior peak and no plateau, and the curve implies a threshold below 0.01 — trading every market — would score marginally better still. The honest summary is that no threshold in the swept range is profitable and performance degrades monotonically as the threshold widens. The two candidates showing large positive returns, $\theta = 0.075$ and $\theta = 0.100$, did so on 4 trades and 1 trade respectively and were excluded by the minimum-trade constraint; we show them for completeness but they are noise, and they demonstrate the constraint is doing real work, since either would otherwise have been selected on essentially no evidence.

Figure 2: Calibration curve on the validation split. Perfect calibration lies on the diagonal.

Trading Performance

The central result of this section is the buy-and-hold row. Backing the market's favored side at the open has a genuinely positive edge *before* costs, earning \$20.87 gross per \$1,000 deployed at a 57.3% win rate — and still loses money, because fees of \$30.73 exceed that edge by half again. This is our proposal's thesis measured rather than asserted: a strategy can be directionally right more often than not, and the fee structure alone converts it into a loss.

The XGBoost policy performs worse than buy-and-hold by a factor of 3.5, and the reason follows directly from the forecast result. Because the model is close to a copy of the market price, the disagreement $|\hat{p} - p^{\text{mkt}}|$ is mostly noise, so a threshold of 0.01 fires on that noise while paying essentially the same fees (\$31.27 against \$30.73) as a policy with a real edge. Its gross PnL is itself negative, so this is not an edge eaten by costs but the absence of an edge to begin with — the simplest policy in the study is the best of the three that trade. Neither clears the no-trade floor, which is the outcome we flagged in the proposal as most likely and the reason the floor appears in the table at all.

Both policies enter every market and neither ever exits early, so holding periods sit just under the 60-candle maximum and turnover is exactly 1.0 — one entry, held to free redemption. We report fee drag against net gross PnL throughout: buy-and-hold spends 147.3% of its net gross PnL on fees, while the same figure is undefined for the XGBoost policy, whose net gross PnL is −\$485.19, since there are no profits for fees to be a fraction of. We note that the alternative convention of counting only profitable markets in the denominator would report the same buy-and-hold trades as consuming 6.9% of gross — a factor of 21.5 on identical data — and would report a comfortable-looking 7.0% for a policy whose gross edge is negative. Reconciling this across both halves of the project is a remaining goal.

4.2 Reinforcement Learning

The Q-learning agent was evaluated by running a fully greedy policy ($\epsilon = 0$) over every episode in the held-out set of evaluation markets and logging metrics for the runs:

- `win_rate`: percentage of trades that resulted in a profit. (A 60% win rate means 6 out of 10 trades were profitable).
- `avg_pnl_per_market`: Total profit divided by the number of markets. It shows how well the model performs on an average asset.
- `total_pnl`: Absolute amount of money the model made (if positive) or lost (if negative) across all its trades.

θ	Trades	Net PnL per \$1,000	Win rate	Eligible
0.010	1,343	−\$34.88	55.0%	yes (selected)
0.020	1,313	−\$39.83	54.3%	yes
0.030	729	−\$43.97	60.1%	yes
0.040	161	−\$53.80	59.6%	yes
0.050	55	−\$187.46	50.9%	yes
0.075	4	+\$462.65	75.0%	no
0.100	1	+\$1,005.12	100.0%	no
0.150	0	—	—	no
0.200	0	—	—	no

Table 3: Validation PnL after fees and slippage across the threshold grid. Only candidates producing at least 30 trades are eligible for selection.

Policy	Gross PnL	Fees	Net PnL	Win rate
No-trade floor	\$0	\$0	**\$0**	—
Buy-and-hold	+\$20.87	−\$30.73	−\$9.86	57.3%
XGBoost, $\theta = 0.01$	−\$3.61	−\$31.27	−\$34.88	55.0%

Table 4: Validation performance per \$1,000 of deployed capital, after fees and slippage, over 1,343 markets with \$100 staked per entry.

- `pnl_per_1k_capital`: Shows exactly how much profit (or loss) was generated for every \$1,000 of starting capital.
- `sharpe_ratio`: How much much excess return we are getting for the extra volatility we are enduring (Sharpe ratio > 1 is generally considered good, > 2 is very good, and > 3 is excellent).
- `max_drawdown`: Largest single drop in portfolio value from a peak to a trough before a new peak is reached.
- `turnover`: How frequently the portfolio is bought and sold. High turnover means the model is making many trades or flipping its entire portfolio rapidly.
- `total_fees_paid`: Absolute amount of money spent on transaction fees, commissions, or exchange costs.
- `fee_fraction_gross_pnl`: Percentage of the raw profits (gross PnL) that was eaten up by trading fees. If a model makes \$100 but pays \$40 in fees, the fraction is 40%.

To see the variance instead of relying on a single lucky or unlucky run, we repeated the full training-and-evaluation pipeline 30 times on two configurations. One was the 70/20 train-evaluation split and the second was the 80/20 train-evaluation split.

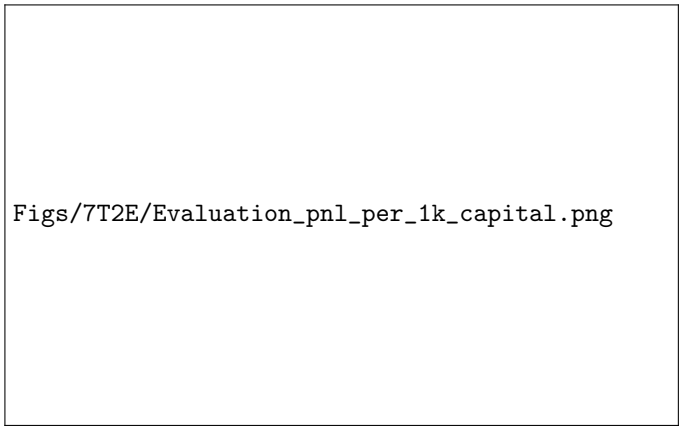
For both the configurations, the agent converges to a policy that is selective but not profitable given the amount paid in fees. The win rate is well above a 50% coin flip in every run, averaging about 65.9% for the 70/20 split and 64.4% in the 80/20 split, which shows the agent reliably picking the side that ends up correct more often than not. However, the average PnL per market is negative in nearly every run once fees are included −\$0.0039 per market on average (70/20) and −\$0.0074 per market (80/20), translating to a mean total PnL of −\$6.75 and −\$12.60 respectively across the held-out markets in each run. Of the 30 held-out runs per configuration, only 3 of the 30 70/20 configuration and 0 out of the 30 80/20 configuration finished with a positive total PnL. The Sharpe ratio in both settings (−0.027 and −0.049) was negative. Figure 3 shows the distribution of total held-out PnL across the 30 independent runs for each split and Figure 4 shows the corresponding win-rate distribution.

The gap between a consistently above-50% win rate and a consistently negative PnL is because of the Polymarket fee structure. On average, fees consumed 24.6% (70/20) and 25.4% (80/20) of gross PnL across runs, which is large enough to erase the small per-trade edge the win rate implies. This is consistent with the underlying markets being close to efficiently priced since a directional bet at price p has an expected payout close to p itself, so the most achievable edge is thin and gets absorbed by the taker fee on both the entry and exit parts of a trade. We also observe that training and held-out evaluation performance track each other closely rather than diverging (e.g. avg PnL per market of −0.0048 training vs. −0.0039 held-out for the 70/20 split, and −0.0054 vs. −0.0074 for the 80/20 split, shown in Figure 5), which shows the agent is not overfitting to specific training-set price paths, it can generalize, it's just not a profitable generalization. The average holding period was essentially identical across every run in both configurations (59.9 of a maximum 60 candles), showing the learned policy almost always rides a position to forced settlement rather than exiting early with `SELL`.

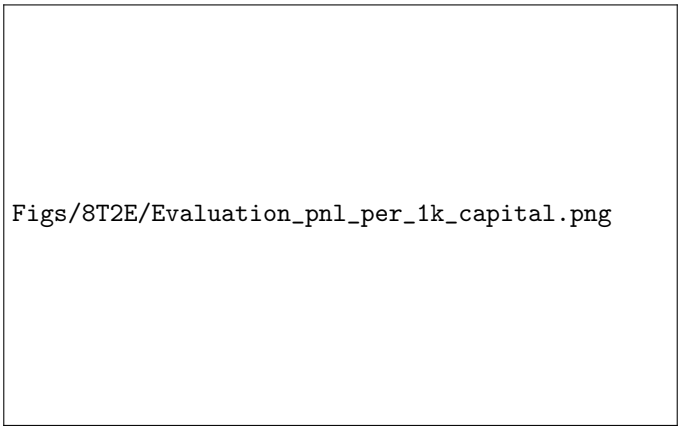
Policy	Sharpe	Max drawdown	Avg. holding (candles)	Fills
No-trade floor	—	\$0	—	0
Buy-and-hold	−2.08	\$4,423	58.8	1,343
XGBoost, $\theta = 0.01$	−7.00	\$7,620	57.5	1,343

Table 5: Extended trading metrics. Sharpe is annualized by scaling per-market returns by $\sqrt{35{,}040}$, the number of 15-minute markets in a year, with no risk-free rate. Max drawdown is in dollars against \$134,300 of deployed capital, not a percentage.

Figure 3: Q-learning: PnL per \$1,000 Capital



(a) 70/20 Split



(b) 80/20 Split

4.3 Rule-Based Baselines

At zero assumed slippage, both rule-based policies from Section 2.4 show a positive edge over the 8,548–8,555 markets they trade (Table 6), and `momentum_flip` looks statistically real on its face: a bootstrap t-statistic of 5.29 against zero edge.

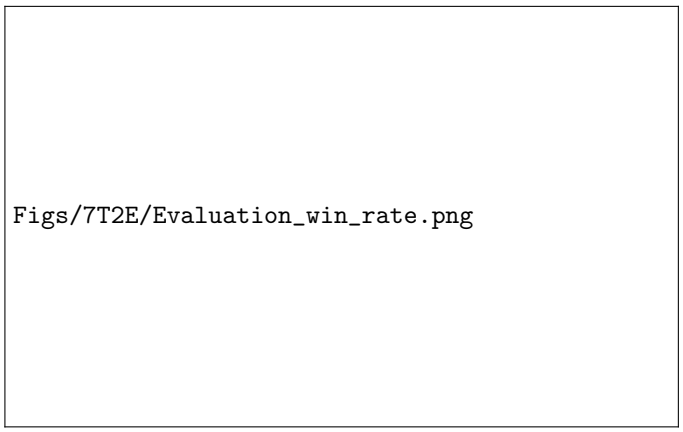
	momentum_flip	buy_and_hold_down
Markets traded	8,548 (99.9%)	8,555 (100%)
Total P&L on \$855,500 staked	+\$28,920	+\$3,192
Avg. return per market	3.38%	0.37%
95% CI (bootstrap)	[2.14%, 4.62%]	[−1.80%, 2.39%]
Win rate	52.6%	50.1%
Profit factor	1.14	1.01
Sharpe per market	0.057	0.004
t-stat vs. zero edge	5.29	0.34
Break-even slippage	\$0.0065/share	\$0.0019/share

Table 6: Rule-based baseline performance at zero slippage, full 8,555-market tape.

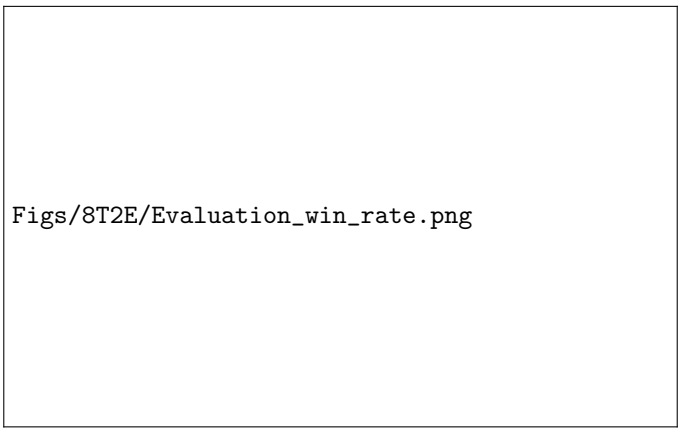
`buy_and_hold_down` is exactly the coin flip it should be (50.1% win rate, t = 0.34), reflecting only the mild Down skew of the sample. `momentum_flip`'s apparent edge does not survive scrutiny for three reasons. First, broken out by how many times a market's position flipped, all of the profit sits in the 2,708 markets that never flipped at all (+\$181,623 combined); markets with 3 or more flips lose money on average, so the flipping rule itself is value-destructive and there is no way to know in advance which markets will not flip. Second, break-even slippage is \$0.0065/share, but the median candle high-low on this book is \$0.04 - an order of magnitude wider - and the full sweep (Table 7) shows both strategies underwater by 0.01 of adverse fill, well inside the book's realistic spread. Third, because a signal fills one candle after it fires, roughly 3% of fills land more than 5 cents from the price that triggered them in either direction; the single best market in the sample (+756%) is one of these - a favorable execution accident, and not the rule working.

Taken together, this track's conclusion agrees with Section 4.1's: the zero-slippage / mid-price view of this market is systematically misleading, and any rule that looks profitable before costs needs to survive a slippage sweep, not just a t-test, before it means anything in refernce to real world performance,

Figure 4: Q-learning: Win Rate

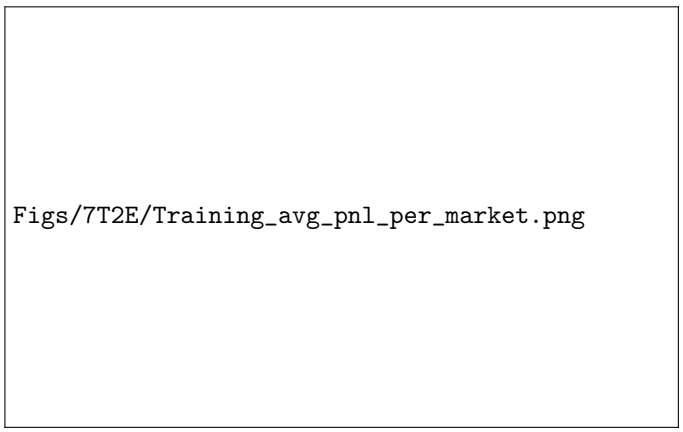


(a) 70/20 Split

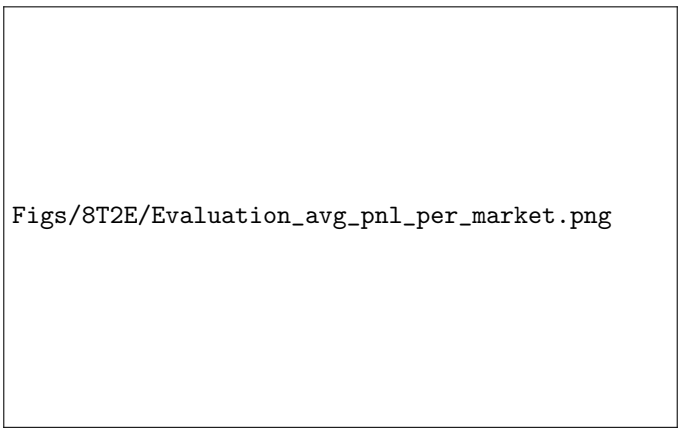


(b) 80/20 Split

Figure 5: Q-learning: Training vs. Evaluation Average PnL per Market



(a) Training



(b) Evaluation

5 Resources Used

- `numpy` for the Q-table and all numerical operations.
- `pandas` for episode/market data manipulation.
- `matplotlib` for training curves, action-frequency plots, and evaluation histograms.
- Gymnasium's `reset/step` API convention (`obs, reward, done, info`) as the pattern for `TradingEnv` in the simulation environment script.
- Polymarket's documentation of its CLOB taker fee schedule to model transaction costs realistically.
- HuggingFace's Q-Learning Course Page.
- ML Compiled's TD Learning Page.
- `xgboost` for the gradient-boosted probability model, using its native early-stopping API against a held-out validation set.
- `scikit-learn` and `scipy` for the calibration and discrimination metrics.
- `pytest` for the 146-test suite covering the cost model, features, temporal splitting, the backtest harness, and the metrics.

Assumed slippage	momentum_flip	buy_and_hold_down
0.000	+$28,920	+$3,192
0.002	+$19,657	−$258
0.005	+$6,092	−$5,382
0.010	−$15,677	−$13,786
0.020	−$56,305	−$30,102

Table 7: Total P&L across assumed adverse fill (slippage), full tape.